首发！建议你一定要看的《AI 生成代码在野安全风险研究》

腾讯技术工程 2025年12月19日 01:46 广东 113人



作者：悟空代码安全团队

随着 AI 技术加速融入软件研发流程，AI 已经成为新时代的“代码贡献者”。在显著提升研发效率的同时，AI 生成代码也对软件安全、漏洞治理和工程责任边界提出了新的挑战。

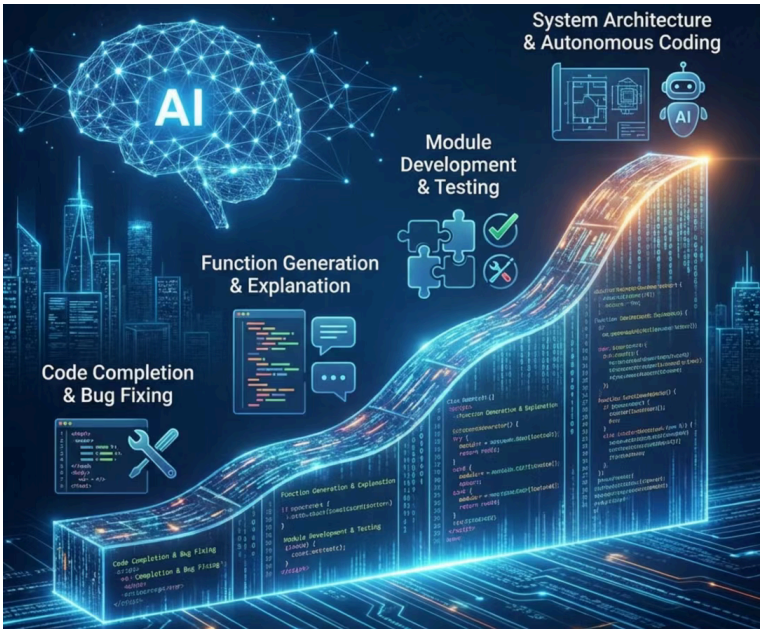
围绕“AI 生成代码在真实世界中的表现与安全性”这一问题，腾讯安全平台部悟空代码安全团队联合北京大学 Narwhal-Lab 与复旦大学系统软件与安全实验室，基于开源项目与真实漏洞（CVE）数据，开展了系统性的实证研究，分析 AI 生成代码的使用趋势、安全风险及其在漏洞生命周期中的角色演变，并提出面向工程实践的治理思路。

文末有礼（含Q币抽奖及完整研究报告PDF）

腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
690篇原创内容
公众号

引言

近年来，随着大语言模型和生成式 AI 技术的快速发展，AI 在编程领域的能力正不断跃迁，从最初的智能补全、模板生成，逐步走向能够理解代码逻辑、优化结构甚至自动生成模块级代码的阶段。AI 编程工具已经深入融入开发者的日常工作流程，在提高生产力和工程效率方面展现出实质性落地效果，越来越多的工程团队在 IDE、代码审查和测试环节中将其作为重要助力，甚至推动了业界广泛关注的“vibe coding”等开发实践趋势。



AI编程能力演变历程

当 AI 生成代码从“效率工具”走向规模化落地，其引入的安全问题也随之被放大，并在真实项目中多次显现，已成为软件安全生态中的潜在“灰犀牛”。在此背景下，行业与学术界陆续对 AI 生成代码的安全性展开针对性研究与评估：

● 工业界方面，Veracode 通过分析 100 个大模型在 80 个精心设计的编程任务上的生成结果发现，尽管 AI 能够稳定产出功能正确的代码，但其生成结果在约 **45% 的情况下引入了安全漏洞**，反映出功能正确性与安全性之间仍存在明显差距。

● 学术研究方面，近年来国内外陆续出现了一批围绕 AI 生成代码安全性的研究工作（如 A.S.E、SUSVIBES 等），通过贴近真实世界编程场景的任务设计，对主流 LLM 与 Agent 在多文件、跨上下文的开发流程中进行评估，同样观察到 **AI 在真实开发场景下生成代码的安全性整体低于预期。**

这些研究共同表明，随着 AI 更深度地参与软件开发流程，**AI 生成代码的安全表现已成为一个无法回避、且亟需被系统性理解的问题。**然而，在真实世界中，AI 生成代码究竟如何被使用、如何演化，以及如何影响软件的安全性，仍缺乏系统性的量化分析。

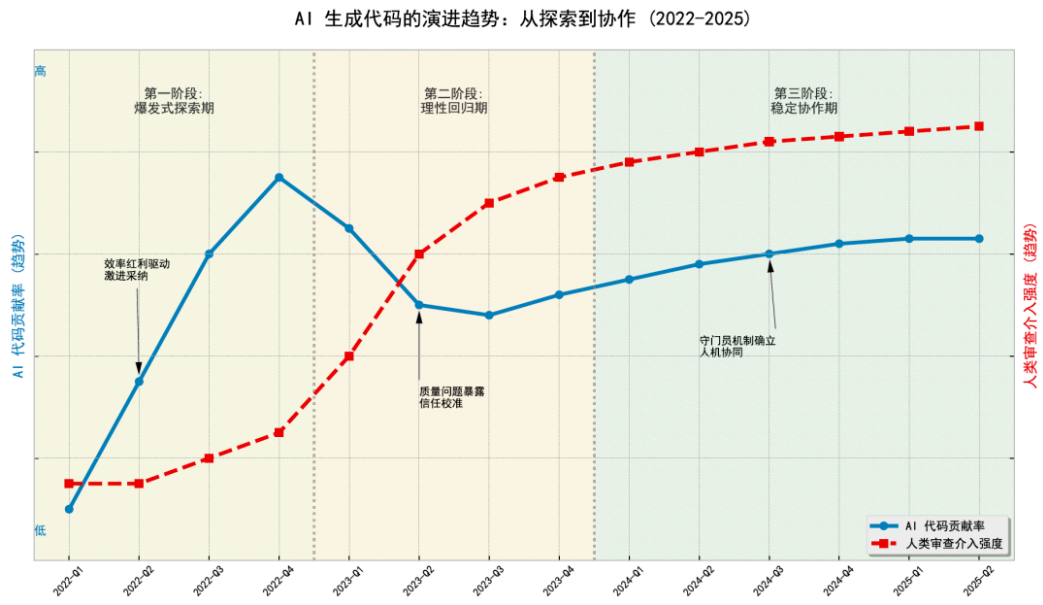
因此，我们开展了本次联合研究，尝试从真实世界视角对该问题进行系统性刻画。**依托开源生态与真实漏洞数据**，我们对 AI 生成代码在实际项目中的使用情况进行分析，重点关注其在不同开发阶段中的分布特征、演化方式以及在漏洞生命周期中的角色变化，并总结其在工程实践中可能带来的影响与启示。

一、 AI生成代码的趋势与特点

1.1 AI生成代码的演进趋势

基于对 GitHub Top 1000项目历史 commit 记录的系统性分析，我们观察到，AI 生成代码在开源生态中的渗透并非线性增长，而是呈现出明显的阶段性演进特征。这一过程既反映了模型能力的

持续提升，也折射出软件工程实践对新技术的适应与调整路径。



真实开发场景 AI 生成代码演进轨迹

阶段 1：爆发式探索期

在 AI 辅助编程工具进入开发者视野的早期阶段，开源社区经历了一段明显的快速扩散期。受益于 IDE 插件等低门槛形态，AI 生成代码在短时间内被广泛尝试，并在新增代码中占比快速上升。开发者倾向于在多类编程任务中直接采纳 AI 生成结果，将其视为提升开发效率的重要手段。

阶段 2：理性回归期

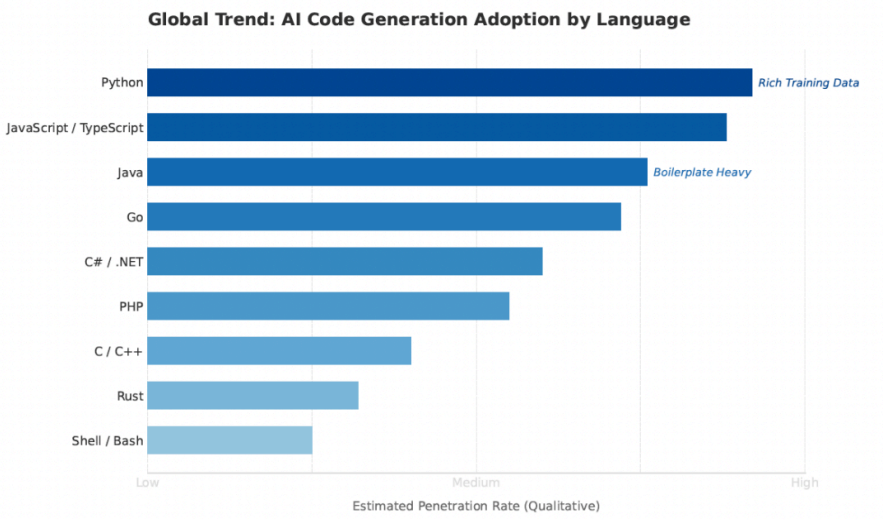
随着使用场景的深入，行业逐步进入理性调整阶段。随着代码规模扩大和工程复杂度提升，AI 在长上下文理解、领域语义把握及非功能性需求上的局限性开始显现，部分场景下的调试与维护成本逐渐放大。相应地，开发者对 AI 的使用开始趋于审慎，在高风险、高复杂度或核心架构相关的代码中，AI 生成内容的占比出现阶段性回调。

阶段 3：稳定协作期

进入近期阶段，AI 生成代码的整体规模在调整后趋于稳定，其在软件开发流程中的角色也发生了结构性变化。AI 不再被简单视为替代开发者的自动化工具，而是逐步融入成熟的人机协作模式：更多承担高重复性、模式化、易验证的任务（如测试代码、文档等），而由人类开发者主导系统设计、业务决策与安全把关。与此同时，数据还显示，AI 在代码维护与缺陷修复等“后期阶段”的参与度持续提升，表明其影响正从代码生成扩展至软件生命周期的更多环节。

1.2 AI 生成代码编程语言分布

AI 代码生成技术在不同技术栈中的渗透程度并不均衡，而是呈现出明显的层次差异。这种差异主要受到两个因素影响：一是相关语言在开源生态中的训练语料规模，二是语言本身的语法特性与开发范式。



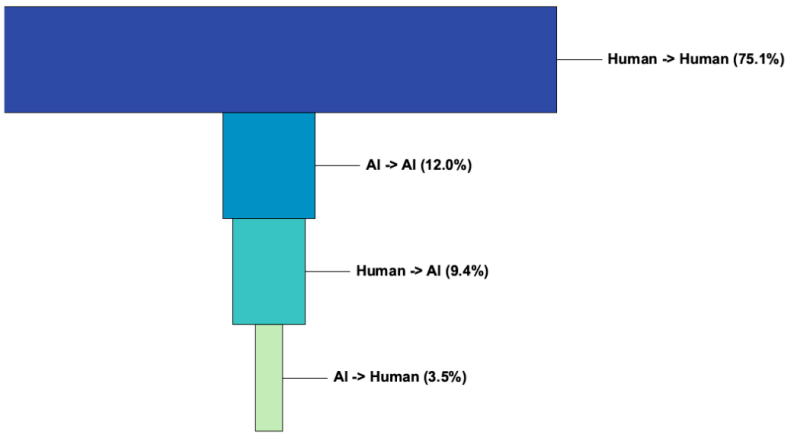
AI 生成代码在不同编程语言中的相对比例

在整体层面，Python、JavaScript、TypeScript 等拥有庞大开源生态的语言，构成了 AI 生成代码的主要来源。这类语言语法灵活、容错性较高，AI 更容易生成较完整的业务逻辑代码。其次是 Java、Go 等企业级语言，尽管其语义和类型约束更严格，但由于存在大量结构化、重复性的样板代码，开发者普遍倾向于借助 AI 来提升开发效率。相比之下，在 Rust、C++ 等系统级语言中，受限于更严格的内存安全和生命周期约束，AI 生成代码的采纳程度明显较低，开发者在这些场景下通常采取更为谨慎的人机协作方式。

二、AI 生成代码的安全风险与漏洞特征

2.1 漏洞生命周期中 AI生成代码的角色

AI 代码生成技术不仅影响代码的生产阶段，也正在深度介入漏洞的发现、修复与演化过程。基于已披露 CVE 漏洞直接相关的修复 commit 记录，我们分析了 AI 生成代码在漏洞修复前后的变化情况。



漏洞修复前后 AI 生成代码变化统计

在3.5%漏洞修复中，原本由 AI 生成的代码，会被开发者用人工代码替换掉。也就是说，当漏洞被定位并进入修复阶段时，开发者会意识到现有实现存在问题，并选择回到人工实现，以获得更

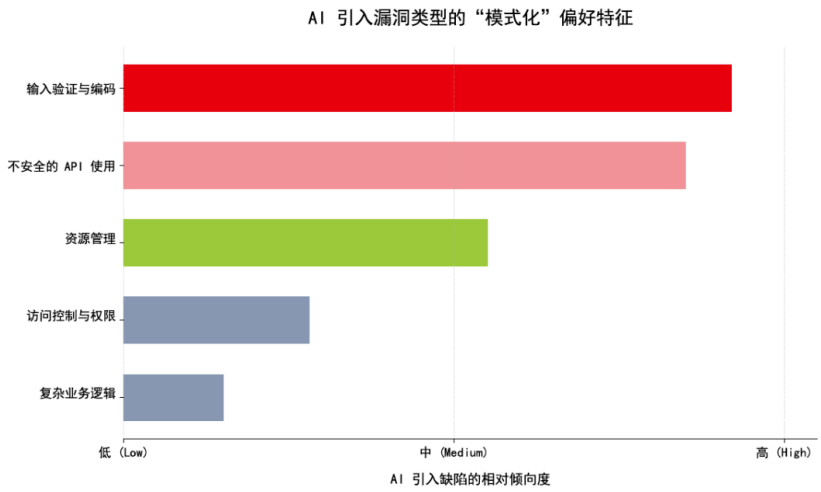
高的可控性和确定性。这类“AI 转人工”的变化，说明在一些安全敏感场景下，AI 生成代码可能更容易成为问题的来源，需要通过人工重写来消除隐患。

与上述现象形成对比，另一类显著趋势表明，AI 生成代码在漏洞修复过程中也扮演了积极的辅助角色。在9.4%的CVE 修复案例中，大量原本由人工编写的代码在修复后被转换为AI 生成代码。这种“人工转AI”的模式，体现了开发者正在借助 AI 工具，加速完成修复逻辑的实现，减少重复性劳动，从而更快响应安全问题。

这两种变化方向同时存在，说明 AI 在漏洞生命周期中的角色并不是固定的。**它既可能成为漏洞产生的一部分，也可能成为漏洞修复过程中的重要助手。**关键不在于“用不用 AI”，而在于“怎么用 AI”。当 AI 被用于高风险逻辑且缺乏充分校验时，问题更容易暴露；而当 AI 被用于辅助重构，并配合人工审查和验证时，其效率优势就会被充分放大。

2.2 AI生成代码引入的漏洞特征画像

与人类开发者因复杂的业务逻辑疏忽或需求理解偏差而导致的漏洞不同，AI 生成代码所引入的缺陷表现出强烈的“模式化”特征。这种特征本质上源于大语言模型对训练数据中历史编码模式的机械模仿，而非对安全原则的深层理解。



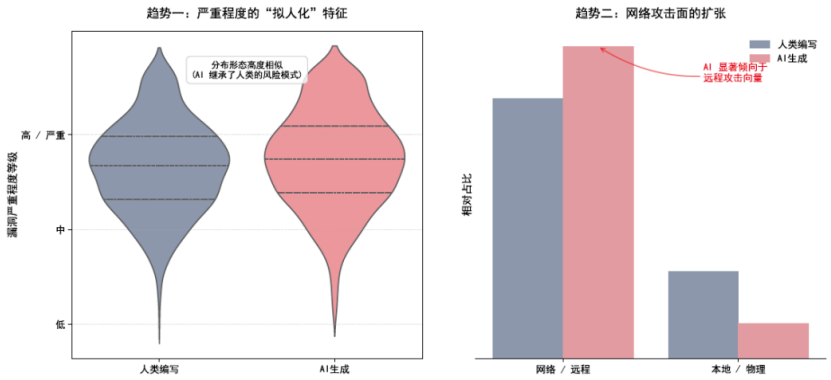
AI 生成代码引入漏洞类型偏好

我们的实证研究表明，AI 引入的漏洞并不是随机分布的，而是高度集中在少数几类常见场景中。其中最突出的两类是：输入验证与数据处理不当，以及 不安全的 API 调用。例如，在处理外部输入、数据序列化或输出格式化时，AI 往往难以准确理解当前的安全上下文，容易遗漏必要的校验、转义或清洗逻辑，从而引入注入类风险。其次，在涉及密码学或敏感操作时，AI 更倾向于复用训练数据中常见但已经过时的实现方式，调用不再符合当前安全标准的算法或接口。

值得注意的是，AI 引入的漏洞极少涉及深层的业务逻辑错误。相反，它们更多体现为代码片段层面的、局部的实现瑕疵。这种“浅层但高频”的分布特征表明，尽管AI 引入了新的风险源，但这些风险在很大程度上是可以通过成熟的静态分析工具与自动化扫描规则进行有效识别和拦截的。

此外，我们进一步分析发现，AI 引入漏洞的风险特征并不“低级”，而是呈现出“严重度拟人化”与“攻击面网络化”的双重趋势。从漏洞严重程度来看，AI 生成代码引入的安全风险并不低于人类开发者。统计结果显示，AI 引入漏洞的严重等级分布与人工代码高度一致，其风险上限同样覆盖可能导致系统不可用或数据泄露的高危场景，表明 AI 生成代码并不存在天然的安全“免疫”或风险

降级特性。在攻击向量维度上，AI 引入的漏洞呈现出更明显的网络侧集中趋势，尤其聚焦于 API 接口、Web 服务交互及网络协议相关代码。这类缺陷通常可通过远程请求触发，无需本地权限，从而在一定程度上扩大了系统的远程攻击面。



AI 生成代码引入的风险特征分析

三、 AI生成代码安全风险缓解建议

针对 AI 代码生成技术的内生缺陷及其带来的软件安全挑战，单一防御手段已难以覆盖其风险边界，需要从模型接入、代码生成到工程落地的全流程入手，围绕评测机制、模型本体能力提升和人机协同治理等多维度协同发力，构建贯穿软件生命周期的多层安全防护体系。



AI 生成代码安全风险缓解框架

维度一：建立多维度的评测基准

由于大语言模型输出具有概率性和非确定性特征，传统安全分析方法难以全面刻画其安全边界。在模型引入前，有必要建立面向 LLM 的代码生成安全评测机制，作为第一道安全防线。行业内目前已有较为成熟的开源实践，例如A.S.E评测集，模拟真实世界 AI 生成代码过程，基于通用缺陷列表（CWE）构建各类安全场景的对抗性测试集，量化评估模型在多语言、多场景环境下的生成安全性。

企业应当在模型引入阶段建立基于此类成熟基准的“红队测试”与准入机制，通过严格的基线测试与持续的攻防演练，确保上线模型具备足够的鲁棒性，从源头上减少存在原生安全缺陷的模型应用

维度二：增强模型本体安全性

提升模型本体的安全性是解决代码幻觉、逻辑漏洞与知识截断问题的根本途径，需要从训练对齐与推理增强两个关键阶段入手。

- 在训练阶段，应强调源头治理，引入包含安全编码规范的高质量数据集并进行严格的数据清洗。利用人类反馈强化学习（RLHF）技术进行深度的安全对齐，引导模型内化安全编程范式，促使模型像人类资深专家一样形成对安全编码的“肌肉记忆”。

- 在推理阶段，针对模型训练数据的知识滞后性问题，建议采用检索增强生成（RAG）技术，动态挂载最新的安全漏洞情报作为外部知识源，使模型在代码生成过程中具备实时风险规避能力。同时，可结合受限解码（ConstrainedDecoding）技术，从抽象语法树（AST）层面强制约束模型的输出结构，阻断非安全API 的调用与危险代码模式的生成。

维度三：人机协同治理

在当前技术条件下，AI 尚无法实现零误报和零风险，人工审查仍是安全闭环中不可替代的一环。治理层面需要重构适应 AI 辅助编程的开发规范，明确开发者对生成代码的最终安全责任。对于 AI 自动生成的代码，应加强可追溯管理，通过元数据标记或隔离执行等方式提升透明度；在审查过程中，将关注重点从语法和风格检查，转向业务逻辑一致性、数据流和权限边界等关键安全属性，并结合静态与动态分析手段识别潜在风险，确保最终安全决策始终由人工把控。

总体而言，AI 生成代码的安全治理是一项系统工程。评测基准提供客观度量，模型本体安全夯实技术基础，人机协同治理构筑最后防线。三者协同作用，才能在充分释放 AI 编程效率的同时，控制其引入的安全风险，推动软件工程向更加可靠和可持续的方向发展。

四、 写在最后：迈向安全可控的 AI 编程未来

AI 参与代码生成已成为现代软件工程发展的既定趋势。通过与北京大学、复旦大学的联合研究，我们更加清晰地看到，AI 生成代码在显著提升开发效率的同时，放大了既有的软件工程复杂性与安全边界。如何通过合理的工程实践与治理体系，引导 AI 能力在可控前提下发挥，正在成为影响其长期价值的关键。

从安全研究的视角来看，未来的关注重点不能只停留在对 AI 生成代码“是否安全”的简单判断，而应转向如何通过工程化手段理解其行为特征，并将其系统性地纳入可治理的开发流程之中。安全不应被视为孤立问题，而应与评估方法、工程流程以及人机协作方式协同演进。在真实世界数据的支撑下，并以工程实践为落点，构建合理的安全评估机制、清晰的流程设计和成熟的人机协作模式，使 AI 在擅长的环节充分释放效率优势，同时通过可验证、可迭代的机制实现持续校准，从而推动 AI 生成代码逐步融入一个更高效、更可靠、也更可持续的软件开发生态。

更多数据分析与完整结果，欢迎点击链接下载《AI 生成代码在野安全风险研究报告》全文

参考文献

[1].https://www.businesswire.com/news/home/20250730694951/en/AI-Generated-Code-Poses-Major-Security-Risks-in-Nearly-Half-of-All-Development-Tasks-Veracode-Research-Reveals

[2].A.S.E: A Repository-Level Benchmark for Evaluating Security in AI-Generated Code

[3].Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks

🎁 关注有礼

关注【腾讯技术工程】账号

可随机抽取5位同学送出腾讯视频VIP月卡

抽奖助手

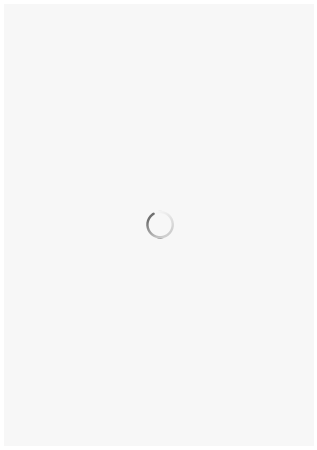
抽奖

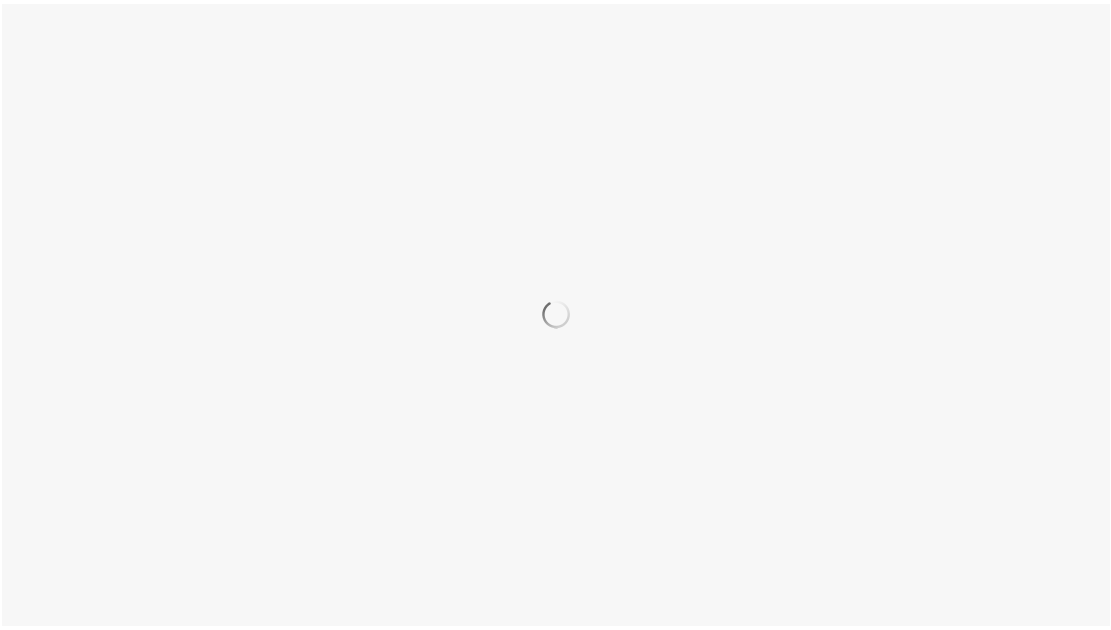


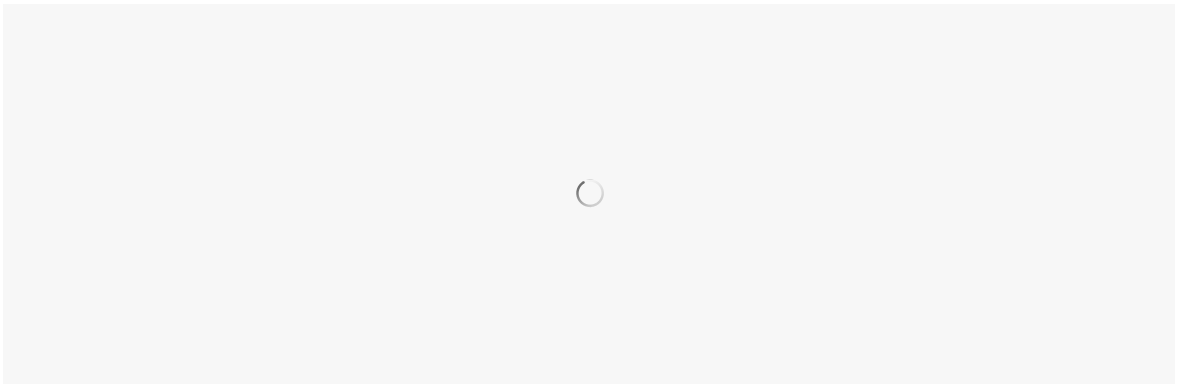
小程序

在【腾讯技术工程】账号后台私信“报告”

即可获取AI生成代码在野安全风险研究报告







悟空 AI Agent · 目录

<上一篇
获NVIDIA致谢：悟空Agent的实践、复盘与迭代

下一篇 >
“龙虾”也需要看病？一张旧病历，引出三个新确诊

阅读 5353

留言

写留言

还没有留言，来写首评